

面向对象程序设计方法（C++）

姓 名:

学 号:

专业班级:

教 师:

时 间:

罗云

2024 年 11 月 07 日

1.1.1 作业 4

1.1.1.1 要求

1. 月薪计算方法是：经理固定工资 8000 元，兼职技术员按 100 元/小时领取月薪，兼职销售员按当月销售额的 4%提成。

2. 工作人员可以通过此程序来管理个人信息，包括查询，添加，修改，删除等功能。

1.1.1.2 评分标准

1. 能正确设计出类及函数，60~70 分；
2. 能实现查询，添加，修改，删除等功能其中 2 个，70~80 分；
3. 全部实现查询，添加，修改，删除等功能，80~90 分；
4. 以上都满足，且设计合理，90~100 分。

1.1.1.3 作业内容

指导老师	罗云		
成绩		教师评语	
关键代码	<pre>// Employee.h class Employee { public: virtual void showInfo() const = 0; virtual double calculateSalary() const = 0; int getId() const { return id; } }; Employee 是一个抽象基类， showInfo() 用于显示员工信息， calculateSalary() 用于计算薪水,每个员工类都必须实现这些方法。</pre>		

```
//Manager.h
```

```
class Manager : public Employee {
```

```
private:
```

```
    double fixedSalary=8000; // 经理固定工资
```

```
public:
```

```
    void showInfo() const override;
```

```
    double calculateSalary() const override;
```

```
    void setFixedSalary(double newSalary) { fixedSalary =  
newSalary; }
```

```
};
```

Manager 类继承自 Employee, 并实现了 calculateSalary() 方法, 返回固定工资 8000 元, showInfo() 方法输出经理的相关信息。

```
// PartTimeTechnician.h
```

```
class PartTimeTechnician : public Employee {
```

```
private:
```

```
    int hoursWorked; // 工作小时数
```

```
    double hourlyRate=100; // 时薪
```

```
public:
```

```
    void showInfo() const override;
```

```
    double calculateSalary() const override;
```

	<pre>void setHoursWorked(int newHours) { hoursWorked = newHours; } void setHourlyRate(double newRate) { hourlyRate = newRate; } }; // PartTimeTechnician.cpp double PartTimeTechnician::calculateSalary() const { return hoursWorked * hourlyRate; // 月薪 = 工时 * 时薪 } PartTimeTechnician 类继承自 Employee , 并实现了 calculateSalary() 方法, 计算月薪 = 工时 × 时薪。 // PartTimeSales.h class PartTimeSales : public Employee { private: double salesAmount; // 销售额 double commissionRate=4; // 提成率 public: void showInfo() const override; double calculateSalary() const override; void setSalesAmount(double newSales) { salesAmount =</pre>
--	--

	<pre>newSales; } void setCommissionRate(double newRate) { commissionRate = newRate; } }; // PartTimeSales.cpp double PartTimeSales::calculateSalary() const { return salesAmount * commissionRate / 100; // 月薪 = 销 销售额 * 提成率 } PartTimeSales 类 继 承 自 Employee , 并 实 现 了 calculateSalary() 方法, 计算月薪 = 销售额 × 提成率。 //main.cpp ● 查询员工 Employee* findEmployeeById(vector<Employee*>& employees, int id) { for (auto emp : employees) { if (emp->getId() == id) { return emp; } } return nullptr; } ● 添加员工: 根据用户输入选择不同类型的员工 (经理、兼职技术 员、兼职销售员), 并使用 baseId 自动递增员工编号。 if (choice == 1) { string name;</pre>
--	--

```

double salary;
cout << "请输入经理姓名: ";
cin >> name;
employees.push_back(new Manager(name, baseId++));
}
else if (choice == 2) {
    string name;
    int hours;
    double rate;
    cout << "请输入技术员姓名: ";
    cin >> name;
    cout << "请输入工作小时数: ";
    cin >> hours;
    employees.push_back(new PartTimeTechnician(name, baseId++,
hours));
}
else if (choice == 3) {
    string name;
    double sales, rate;
    cout << "请输入销售员姓名: ";
    cin >> name;
    cout << "请输入销售金额: ";
    cin >> sales;
    employees.push_back(new PartTimeSales(name, baseId++, sales));
}

```

● 修改员工信息:

```

else if (choice == 6) {
    int id;
    cout << "请输入员工编号修改信息: ";
    cin >> id;
    Employee* emp = findEmployeeById(employees, id);
    if (emp) {
        cout << "修改前信息:" << endl;
        emp->showInfo();
        // 根据员工类型修改信息
        int subChoice;
        cout << "1. 修改姓名\n2. 修改工资信息\n请选择: ";
        cin >> subChoice;
        if (subChoice == 1) {
            string newName;
            cout << "请输入新姓名: ";
            cin >> newName;
            emp->setName(newName);

```

	<pre> } else if (subChoice == 2) { if (dynamic_cast<Manager*>(emp)) { double newSalary; cout << "请输入新的固定工资: "; cin >> newSalary; dynamic_cast<Manager*>(emp)->setFixedSalary(newSalary); } else if (dynamic_cast<PartTimeTechnician*>(emp)) { int newHours; double newRate; cout << "请输入新的工作小时数: "; cin >> newHours; cout << "请输入新的时薪: "; cin >> newRate; dynamic_cast<PartTimeTechnician*>(emp)->setHoursWorked(newHours); dynamic_cast<PartTimeTechnician*>(emp)->setHourlyRate(newRate); } else if (dynamic_cast<PartTimeSales*>(emp)) { double newSales, newRate; cout << "请输入新的销售额: "; cin >> newSales; cout << "请输入新的提成率: "; cin >> newRate; dynamic_cast<PartTimeSales*>(emp)->setSalesAmount(newSales); dynamic_cast<PartTimeSales*>(emp)->setCommissionRate(newRate); } } } else { cout << "员工未找到!" << endl; } } ● 删除员工 else if (choice == 7) { int id; cout << "请输入员工编号删除: "; cin >> id; </pre>
--	--

	<pre>auto it = find_if(employees.begin(), employees.end(), [id](Employee* emp) { return emp->getId() == id; }); if (it != employees.end()) { delete* it; employees.erase(it); cout << "员工已删除!" << endl; } else { cout << "员工未找到!" << endl; }}</pre>
--	--

添加员工信息

1. 添加经理

2. 添加兼职技术员

3. 添加兼职销售员

4. 显示所有员工

5. 查询员工

6. 修改员工信息

7. 删除员工

8. 退出

请输入您的选择：1

请输入经理姓名：pp

1. 添加经理

2. 添加兼职技术员

3. 添加兼职销售员

4. 显示所有员工

5. 查询员工

6. 修改员工信息

7. 删除员工

8. 退出

请输入您的选择：2

请输入技术员姓名：pp

请输入工作小时数：50

1. 添加经理

2. 添加兼职技术员

3. 添加兼职销售员

4. 显示所有员工

5. 查询员工

6. 修改员工信息

7. 删除员工

8. 退出

请输入您的选择：3

请输入销售员姓名：pp

请输入销售金额：5000

修改员工信息

请输入您的选择：6

请输入员工编号修改信息：1000

修改前信息：

经理姓名：pp，员工编号：1000，固定工资：8000

1. 修改姓名

2. 修改工资信息

请选择：2

请输入新的固定工资：5000

```
请输入您的选择：6
请输入员工编号修改信息：1001
修改前信息：
技术员姓名：pp，员工编号：1001，工时：50，时薪：100
1. 修改姓名
2. 修改工资信息
请选择：2
请输入新的工作小时数：100
请输入新的时薪：100
```

```
请输入您的选择：6
请输入员工编号修改信息：1002
修改前信息：
销售员姓名：pp，员工编号：1002，销售额：5000，提成率：4
1. 修改姓名
2. 修改工资信息
请选择：
2
请输入新的销售额：10000
请输入新的提成率：101
```

查看员工信息

```
请输入您的选择：4
经理姓名：pp，员工编号：1000，固定工资：5000
月薪：5000
技术员姓名：pp，员工编号：1001，工时：100，时薪：100
月薪：10000
销售员姓名：pp，员工编号：1002，销售额：10000，提成率：101
月薪：10100
```